Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИМЕНИ
ПЕРВОГО ПРЕЗИДЕНТА РОСИИ Б.Н.ЕЛЬЦИНА»
(ИРИТ-РТФ
ЦЕНТР УСКОРЕННОГО ОБУЧЕНИЯ)

ОТЧЕТ
ПО ПРОЕКТНОМУ ПРАКТИКУМУ
АВТОМАТИЗАЦИЯ И РАЗРАБОТКА ПО

Куратор _____ К.А. Аксенов
подпись, дата

Екатеринбург 2026

СПИСОК ИСПОЛНИТЕЛЕЙ

Студент группы
РИЗ-240916у _____ А.В. Масляков
подпись, дата

Студент группы
РИЗ-240916у _____ А.В. Казанцев
подпись, дата

Студент группы
РИЗ-240916у _____ К.Н. Маркин
подпись, дата

Студент группы
РИЗ-240916у _____ Н.В. Палинов
подпись, дата

Студент группы
РИЗ-240916у _____ К.Р. Мансуров
подпись, дата

Студент группы
РИЗ-240916у _____ Е.М. Разорин
подпись, дата

Студент группы
РИЗ-240916у _____ Д.С. Скобелин
подпись, дата

СОДЕРЖАНИЕ

Введение . . . 5

1 Описание предметной области . . . 6

1.1 Ананлиз специфики предметной области . . . 6

1.2 Основные понятия и термины . . . 7

1.3 Обзор близких технических решений . . . 8

2 База знаний . . . 10

2.1 Структура фактов и их семантика . . . 10

2.2 Принципы построения продукционных правил . . . 11

2.3 Формирование правил . . . 12

3 Проверка непротиворечивости . . . 15

3.1 Методы проверки . . . 15

3.2 Результаты анализа . . . 16

3.3 Ограничения и практические замечания . . . 17

3.4 Вывод по разделу . . . 18

4 Средства разработки . . . 19

4.1 Обоснование выбора языка программирования . . . 19

4.2 Описание используемых библиотек . . . 20

4.3 Инструменты разработки . . . 21

5 Техническое решение . . . 22

5.1 Архитектура системы . . . 22

5.2 Схема работы . . . 24

5.3 Алгоритм реализации . . . 26

6 Тестирование прототипа . . . 28

Заключение . . . 30

Список использованных источников . . . 31

Приложение А Исходный код form_factory.py . . . 32

Приложение Б Исходный код forms.py 35
Приложение В Исходный код knowledge.py 36
Приложение Г Исходный код rules.py 39
Приложение Д Документация по использованию 45

ВВЕДЕНИЕ

Туризм остается сложной сферой, где решение клиента зависит сразу от нескольких факторов: цели поездки, условий отдыха, бюджета и состава группы. В такой ситуации человеку трудно быстро выбрать подходящий вариант без структурированной консультации.

Для туристских услуг особенно важна точная привязка предложения к потребностям пользователя. Если такая привязка слабая, рекомендация получается общей и теряет практическую ценность.

Цифровые инструменты в туризме развиваются, и вместе с этим растет запрос на понятные и прозрачные механизмы принятия решений. Для прикладных систем это означает, что важно не только выдать ответ, но и показать логику, по которой он получен [1].

В данной работе эта задача решается через прототип продукционной экспертной системы «Консультант по путешествиям». Выбран подход с явными правилами вида «ЕСЛИ -> ТО», потому что он упрощает проверку рекомендаций и позволяет объяснять результат пользователю.

Практическая значимость проекта состоит в объединении трех функций: структурированный ввод фактов, автоматический выбор рекомендации и explain-режим с прямым и обратным выводом. Такой формат подходит для учебной разработки и для поэтапного расширения базы знаний [2].

Таким образом, тема экспертного консультирования в туризме является актуальной с точки зрения качества сервиса и цифровизации отрасли. Далее в отчете последовательно рассматриваются предметная область, понятийная база и особенности решений, на которых строится прототип [3].

1 Описание предметной области

1.1 Ананлиз специфики предметной области

Разрабатываемое приложение относится к сфере туристско-рекреационных услуг и решает задачу консультирования пользователя по выбору формата поездки. Предметная область туризма рассматривается как социально-экономическая система, в которой взаимодействуют субъект (турист с его мотивами и ограничениями) и объект (туристский регион, предприятия и организации, формирующие услугу) [3,4].

Для экспертной системы это означает, что рекомендация не может строиться только по одному признаку. Решение формируется на пересечении цели поездки, условий поездки и профиля туриста. В материалах по основам и рекреационному туризму подчеркивается, что качество услуги определяется прежде всего достижением основной цели путешествия, а не только уровнем отдельных сервисов [3,5]. Поэтому в базе знаний критична связка «мотив поездки -> тип туристской программы».

С прикладной точки зрения на логику правил влияют четыре группы характеристик предметной области. Первая группа: функции туризма (экономическая, социальная, гуманитарная и воспитательная), которые задают возможные пользовательские запросы от восстановления ресурсов до познавательных и деловых целей [1,4]. Вторая группа: внешние факторы (природно-климатические условия, экономическая ситуация, политико-правовые ограничения, технологическая доступность, экологическая обстановка). Третья группа: внутренние факторы отрасли (уровень инфраструктуры, формат обслуживания, степень дифференциации предложений). Четвертая группа: сезонность, из-за которой меняются доступность и релевантность рекомендаций.

Для прототипа «Консультант по путешествиям» это задает требования к структуре фактов и правил. Факты должны отражать: цель поездки, предпочтительный формат активности, ограничения по условиям пребывания, ожидаемый уровень сервиса и безопасности. Правила должны классифицировать запрос в один или несколько классов программного туризма (оздоровительный, познавательный, деловой, экологический, спортивный, обучающий, семейный), после чего выдавать целевую рекомендацию по типу тура и формату обслуживания [3,5].

Отдельно значима компонента безопасности: при проектировании туристских услуг учитываются риски среды, физические нагрузки, биологические угрозы и личная безопасность. Для экспертной системы это выражается в необходимости включать в консультацию хотя бы базовые предупреждения и условия применимости рекомендации к профилю туриста [2,4].

1.2 Основные понятия и термины

- туризм: путешествие и временное пребывание вне обычной среды, не связанное с оплачиваемой деятельностью в месте пребывания;
- турист: ночующий посетитель, совершающий поездку вне обычной среды в установленных временных пределах;
- посетитель: общее понятие для лиц, путешествующих вне обычной среды; включает туристов и экскурсантов;
- экскурсант: однодневный посетитель без ночевки в месте посещения;
- внутренний туризм: поездки в пределах собственной страны вне обычной среды;
- международный туризм: поездки за пределы страны постоянного проживания вне обычной среды;
- программное обслуживание: целевой комплекс услуг, объединенных мотивом поездки и ориентированных на конкретного потребителя;
- программа обслуживания: структурированный по времени набор услуг, обеспечивающий достижение цели путешествия;
- рекреационная деятельность: деятельность, направленная на восстановление физических и психоэмоциональных ресурсов человека;
- мотивация в туризме: совокупность причин, определяющих выбор направления, формата отдыха и туристской фирмы;
- сезонность: регулярные колебания туристского спроса с выраженными периодами пиков и спадов;

– туристские ресурсы: природные, культурно-исторические и социокультурные объекты, формирующие привлекательность территории;
– туристская инфраструктура: средства размещения, питания, транспорта, досуга и информационного сопровождения, обеспечивающие реализацию турпродукта;
– безопасность в туризме: учет и снижение рисков, связанных с условиями среды, нагрузками, биологическими и социальными угрозами [3].

1.3 Обзор близких технических решений

Сопоставление выполнено по материалам анализа главных страниц `ostrovok`, `turscanner` и `tripster` относительно текущего прототипа «Консультант по путешествиям». Критерии: полнота структурированного ввода, пригодность фактов к продукционным правилам и возможность объяснимого вывода [1,4,5].

Текущая система в проекте ориентирована на консультацию по правилам: пользователь задает формализованные параметры поездки, а модуль правил формирует рекомендацию. Поэтому сравнение аналогов проводилось с позиции применимости к экспертной диагностике, а не только к каталожному поиску. Сравнительные характеристики приведены в 1.

Таблица 1 — Название

Решение	**Сильные стороны**	**Узкие места**	**Сравнение с текущей системой**
Ostrovok	Направление, даты, состав гостей и тип поездки заданы явно.	Слабо выражены мотив поездки, бюджетные ограничения и профиль группы для классификации типа тура.	Близок по структуре ввода, но в меньшей степени ориентирован на экспертный вывод.
Turscanner	Явные фильтры сезонности, города вылета и признака «без визы».	Короткая анкета: нет цели, длительности, состава группы и предпочтений формата отдыха.	Лучше отражает внешние ограничения, но хуже поддерживает персонализированные правила.
Tripster	Быстрый поиск направления с подсказками.	На главной форме почти нет структурированных туристских фактов кроме направления.	Эффективен как навигация по каталогу, но заметно слабее текущего прототипа для экспертной диагностики.

Текущий прототип методически ближе к целевой архитектуре экспертной системы. Из аналогов рационально заимствовать компактность формы (`Ostrovok`) и явные сезонно-визовые фильтры (`Turscanner`) [1].

2 База знаний

База знаний текущей версии прототипа реализована как совокупность формализованных фактов и явных продукционных правил вида «ЕСЛИ (условия), ТО (рекомендация)». В рабочей реализации используются `67` предметных правил и отдельное fallback-правило `default-recommendation`, которое возвращает универсальную рекомендацию, если пользовательский профиль не доказал ни одно специализированное заключение. Такая организация соответствует учебной задаче: логика вывода читается непосредственно в коде, а объяснение результата опирается на проверяемый набор условий [1,3].

При проектировании базы знаний использовался принцип schema-driven-описания фактов. Один и тот же реестр признаков служит источником для построения веб-формы, нормализации пользовательского ввода, формирования факта `TravelInput` и explain-режима. За счет этого изменения в составе фактов выполняются в одной точке и не создают рассинхронизации между интерфейсом и логическим ядром. Такой подход особенно важен для экспертной системы, где ошибки на уровне формулировки признака быстро превращаются в ошибки вывода.

2.1 Структура фактов и их семантика

В продукционных правилах участвуют десять логических слотов. Девять из них задаются пользователем в явном виде, а поле `notes` сохраняется в сессии, но в самом выводе не участвует.

Таблица 2 — Логические слоты базы знаний

Слот	**Смысл**	**Типичные значения**
`season`	Сезон поездки	`spring`, `summer`, `autumn`, `winter`, `any`
`hobby`	Преобладающее туристское увлечение	`hiking`, `museum`, `food`, `any`
`budget_rub`	Бюджет в рублях	Числовой интервал, используемый через пороги `lt`, `lte`, `gt`, `gte`
`trip_days`	Длительность поездки в днях	Числовой интервал от `1` до `60`
`climate`	Предпочитаемый климат	`warm`, `mild`, `cold`, `any`
`travel_type`	Основная цель и формат поездки	`relax`, `active`, `mixed`, `culture`, `health`, `business`, `eco`, `education`
`companions`	Состав поездки	`solo`, `couple`, `family`, `friends`

Слот	Смысл	Типичные значения
service_level	Ожидаемый уровень сервиса	economy, standard, premium
visa_mode	Готовность к визовым процедурам	any, visa_free_only, visa_ready
insurance	Наличие или готовность оформить страховку	yes, no

Состав фактов охватывает четыре содержательных блока. Первый блок описывает контекст поездки: season, climate, visa_mode. Второй блок фиксирует мотивацию и поведенческий профиль: travel_type, hobby. Третий блок задает ресурсные ограничения: budget_rub, trip_days, service_level. Четвертый блок отражает социальные и безопасностные ограничения: companions, insurance. Именно такая структура позволяет переводить короткую анкету пользователя в набор признаков, достаточный для выбора вида турпродукта и уровня обслуживания [4,5].

Семантически важным решением является сочетание категориальных и числовых фактов. Категориальные признаки задают класс сценария, а числовые пороги по бюджету и длительности уточняют глубину рекомендации. Например, различаются условия вида «budget_rub >= 100000» и «trip_days <= 7», что позволяет отделять премиальные и длительные сценарии от базовых. Поддерживаются операторы eq, lt, lte, gt, gte, поэтому числовые ограничения оформлены не как словесные ярлыки, а как воспроизводимые интервальные условия.

2.2 Принципы построения продукционных правил

База правил выстроена иерархически. Верхний слой составляют наиболее специфичные рекомендации с приоритетами порядка 340-309: они учитывают одновременно цель поездки, климат, бюджет, длительность, состав группы, визовый режим, уровень сервиса и наличие страховки. Средний слой (305-270) описывает устойчивые предметные сценарии средней детализации, например семейный отдых в умеренном климате, бюджетную поездку на выходные, короткий культурный тур, стандартный курортный отпуск или экономичный маршрут внутри страны. Нижний слой (260-243) содержит общие правила по классу путешествия: деловой, экологический, обучающий, оздоровительный, культурный, спокойный, активный или смешанный формат [6].

Такое построение поддерживает принцип градации по уточнению. Чем больше значимых ограничений учитывает правило, тем выше его `priority` и тем раньше оно рассматривается при объяснении. В прямом выводе это соответствует `salience` библиотеки `experta`, а в explain-слое используется тот же числовой приоритет. В результате пользовательский профиль может удовлетворять нескольким правилам одновременно, но итоговая рекомендация выбирается детерминированно: побеждает правило с максимальным приоритетом.

Второй принцип состоит в предметной релевантности заключений. Рекомендация формулируется не как абстрактный класс тура, а как прикладной сценарий, привязанный к туристской ситуации. Например, правило «если `climate=warm`, `travel_type=relax`, `budget_rub>=100000`, то рекомендовать пляжный отдых в теплой стране с повышенным уровнем комфорта» уточняет и тип поездки, и ожидаемый уровень сервиса. Аналогично правило «если `season=winter`, `climate=cold`, `travel_type=active`, то рекомендовать зимний активный тур с горнолыжным курортом» связывает сезонность, климат и характер активности в один содержательный вывод [3].

Третий принцип заключается в объяснимости. Каждое правило сопровождается метаданными: именем, приоритетом, текстом рекомендации и формализованным набором условий `ConditionSpec`. Благодаря этому в прямом и обратном выводе система сохраняет не только итог, но и список сработавших правил, выбранное правило и шаги проверки условий. Для учебного прототипа этот аспект принципиален, поскольку позволяет анализировать не только ответ, но и сам процесс рассуждения [2].

2.3 Формирование правил

Формирование базы выполнялось от общего к частному. Сначала были зафиксированы базовые рекомендации по доминирующему типу поездки: `business-general`, `eco-general`, `education-general`, `health-general`, `culture-general`, `relax-general`, `active-general`, `mixed-general`. Эти правила гарантируют, что каждый осмысленный класс запроса имеет хотя бы одну предметную интерпретацию.

Затем к базовому слою были добавлены уточняющие правила по отдельным осям персонализации:

1. по сезонности и климату: «если season=summer и climate=warm, то рекомендовать летний морской отдых в теплом климате»; «если climate=mild и season=spring, то рекомендовать весенний маршрут в умеренном климате»;
2. по компании: «если companions=family и travel_type=relax, то рекомендовать спокойный семейный отдых»; «если companions=friends и travel_type=active, то рекомендовать активную поездку для друзей»;
3. по бюджету и длительности: «если budget_rub<50000 и trip_days<=3, то рекомендовать недорогую поездку на выходные по России»; «если travel_type=active, trip_days>=12, budget_rub>=110000, то рекомендовать активное путешествие экспедиционного типа»;
4. по визовому режиму, страховке и сервису: «если visa_mode=visa_free_only и trip_days<=7, то рекомендовать короткое безвизовое направление»; «если insurance=yes и travel_type=health, то рекомендовать оздоровительную поездку с медицинским блоком»; «если service_level=premium и budget_rub>=120000, то рекомендовать комфортную поездку с премиальным сервисом».

На завершающем этапе были добавлены составные правила высокой специфичности. Именно они формируют наиболее персонализированные ответы, например:

1. если season=summer, climate=warm, companions=family, то рекомендовать семейный летний пляжный отдых;
2. если season=winter, climate=cold, travel_type=active, то рекомендовать зимний активный тур с горнолыжным курортом;
3. если travel_type=business, service_level=premium, budget_rub>=90000, то рекомендовать деловую поездку в крупный город с премиальным сервисом;
4. если visa_mode=visa_free_only, companions=family, climate=warm, то рекомендовать теплое семейное направление без визы;
5. если climate=warm, travel_type=relax, budget_rub>=150000, trip_days>=14, service_level=premium, visa_mode=visa_ready,

`insurance=yes`, то рекомендовать долгий расслабленный отпуск с визой.

Таким образом, база знаний сочетает полноту покрытия и управляемую детализацию. Общие правила отвечают за устойчивость вывода, а специализированные правила повышают персонализацию консультации без разрушения общей структуры приоритетов.

3 Проверка непротиворечивости

Проверка непротиворечивости в текущей версии проекта выполняется как анализ реальной базы из 67 предметных правил и одного fallback-правила, а не как абстрактная проверка «идеи системы». Цель проверки состоит в том, чтобы подтвердить три свойства: отсутствие прямых логических конфликтов, детерминированность выбора рекомендации при множественном срабатывании и полноту покрытия пользовательских профилей [7].

В отличие от ранней версии отчета, актуальная база правил уже содержит формализованный механизм разрешения конкуренции. Каждое правило имеет фиксированный `priority`, а в движке прямого вывода используется тот же ранг как `salience`. Поэтому для одного набора фактов возможна конкуренция нескольких содержательно совместимых правил, но не допускается недетерминированность итогового выбора: побеждает рекомендация с максимальным приоритетом.

3.1 Методы проверки

Проверка непротиворечивости выполнялась по четырем направлениям.

Первое направление: нормализация условий. Для каждого правила фиксировались атомарные условия по слотам `season`, `hobby`, `budget_rub`, `trip_days`, `climate`, `travel_type`, `companions`, `service_level`, `visa_mode`, `insurance`, а также итоговая рекомендация и ее приоритет. Это позволило сравнивать правила не по текстовому виду, а по фактической структуре предпосылок.

Второе направление: поиск прямых формальных конфликтов. Конфликтом считалась пара правил с одинаковой левой частью, но разными рекомендациями. По актуальному каталогу таких пар не обнаружено: правила различаются либо хотя бы одним условием, либо уровнем общности и потому не задают два взаимоисключающих вывода для одного и того же полного набора предпосылок.

Третье направление: анализ отношений «общее правило -> частное правило». В туристской базе знаний это основной

источник множественного срабатывания. Например, правило «если travel_type=active, budget_rub<100000, trip_days<=7, то рекомендовать активный короткий тур по России или соседним направлениям» логически совместимо с правилом «если travel_type=active, то рекомендовать общий активный формат поездки». Здесь нет противоречия: второе правило задает общий класс решения, а первое уточняет его и имеет более высокий приоритет.

Четвертое направление: сценарная проверка explain-логики. Для типовых профилей анализировались одновременно matched_rules и selected_rule, чтобы убедиться, что цепочка рассуждения остается содержательно согласованной. Практически это означает проверку того, что список сработавших правил не содержит взаимоисключающих рекомендаций, а выбранное правило действительно является самым специфичным или наиболее приоритетным среди доказанных.

3.2 Результаты анализа

Проверка показала, что в актуальной версии базы отсутствуют прямые формальные противоречия вида «одно и то же условие -> разные заключения». Основная форма взаимодействия правил в системе иная: одно пользовательское состояние обычно активирует сразу несколько совместимых рекомендаций разной степени общности.

Это хорошо видно на нескольких типовых сравнениях.

1. «Если season=summer, climate=warm, companions=family, то рекомендовать семейный летний пляжный отдых» совместимо с правилом «если climate=warm и season=summer, то рекомендовать летний морской отдых в теплом климате». Второе правило описывает общий сезонно-климатический сценарий, первое конкретизирует его для семейной поездки и закономерно имеет более высокий приоритет.
2. «Если travel_type=business, service_level=premium, budget_rub>=90000, то рекомендовать деловую поездку в крупный город с премиальным сервисом» совместимо с правилом «если travel_type=business, то рекомендовать общий деловой формат

поездки». Здесь частное правило уточняет сервисную и бюджетную конфигурацию общего класса делового тура.

3. «Если visa_mode=visa_free_only, companions=family, climate=warm, то рекомендовать теплое семейное направление без визы» совместимо с правилом «если visa_mode=visa_free_only и trip_days<=7, то рекомендовать короткое безвизовое направление». Эти правила могут пересекаться на части профилей, но они описывают разные уровни детализации: одно акцентирует семейный теплый отдых, другое ограничение по визе и длительности.
4. «Если companions=family, travel_type=health, insurance=yes, то рекомендовать семейную оздоровительную программу» совместимо с правилом «если insurance=yes и travel_type=health, то рекомендовать оздоровительную поездку с медицинским блоком». Общий медицинский сценарий не конфликтует с семейным, а уточняется им.

Во всех таких случаях множественное срабатывание не является логической ошибкой. Напротив, оно отражает нормальную структуру продукционной системы, в которой общие правила обеспечивают полноту покрытия, а специальные правила повышают персонализацию вывода. Непротиворечивость достигается не устранением всех пересечений, а их явной иерархизацией через priority.

Отдельно проверялась полнота базы. Нижний слой общих правил по типу поездки гарантирует, что любой профиль с корректно выбранным travel_type получает как минимум базовую интерпретацию. Если же профиль не доказывает ни одно предметное правило, применяется default-recommendation со значением «Рекомендуется универсальный экскурсионный отдых по вашему бюджету и срокам». Следовательно, база не содержит пустых состояний, в которых система остается без ответа.

3.3 Ограничения и практические замечания

Хотя прямых противоречий в базе не обнаружено, анализ выявил одно важное ограничение покрытия. В каталоге присутствует правило «если hobby=dance и travel_type=culture, то рекомендовать культурную поездку на фестивали и танцевальные мероприятия», однако значение dance

отсутствует в стандартном наборе вариантов поля `hobby` пользовательской формы. Это не противоречие и не источник неверного вывода, но это признак рассогласования между доступным интерфейсом ввода и полной моделью знаний.

Практически это означает, что логическая база сейчас более богата, чем веб-форма. Для методической целостности системы желательно либо добавить значение `dance` в реестр фактов, либо удалить соответствующее правило как недостижимое из штатного интерфейса. Такая правка относится не к непротиворечивости, а к полноте операционализации базы знаний.

3.4 Вывод по разделу

Актуальная база знаний является логически согласованной: прямых противоречий между правилами не выявлено, конкуренция рекомендаций разрешается детерминированно через `priority`, а fallback-правило устраняет риск пустого результата. Основной механизм множественного срабатывания в системе основан не на конфликте, а на отношении «общее правило -> специализированное правило», что соответствует архитектуре продукционной экспертной системы и обеспечивает объяснимый выбор итоговой рекомендации.

4 Средства разработки

Для реализации прототипа экспертной системы использован практико-ориентированный стек средств разработки, обеспечивающий быструю итерацию правил, прозрачность логики вывода и воспроизводимость результата. Выбор инструментов выполнялся с учетом учебного формата проекта, требований к explain-режиму и необходимости сопровождения базы знаний командой разработчиков.

4.1 Обоснование выбора языка программирования

Базовым языком реализации выбран Python. Первое основание выбора связано с выразительностью языка при описании предметных фактов и условий правил. Для экспертной системы критично, чтобы структура знаний оставалась читаемой в исходном коде и легко ревизовалась при расширении базы правил. Синтаксис Python и поддержка декларативных конструкций позволяют формулировать правила и метаданные в форме, близкой к естественной логике предметной области.

Второе основание — технологическая совместимость со стеком веб-части и вспомогательных модулей. Python обеспечивает единый контур для формы ввода, слоя нормализации фактов, движка вывода и сохранения результатов консультации. За счет этого снижаются интеграционные издержки: не требуется разделять решение между разными языковыми платформами и согласовывать межъязыковой обмен данными.

Третье основание — темп разработки и сопровождения. Для учебного прототипа важна короткая итерация «изменение правила -> локальная проверка -> анализ результата». В Python этот цикл минимален благодаря компактной записи бизнес-логики, отсутствию избыточного шаблонного кода и широкому набору стандартных инструментов тестирования.

Четвертое основание — поддерживаемость при командной работе. В проекте база правил регулярно уточняется, а значит, код должен быть понятен разработчику, который не является автором исходного правила. В данном аспекте Python обеспечивает низкий порог входа, предсказуемую структуру модулей и простой формат ревью изменений.

Пятое основание — приемлемый баланс производительности и сложности. Хотя интерпретируемый язык уступает низкоуровневым решениям в скорости, для прототипа с ограниченным числом правил и однопользовательским сценарием это не является критическим ограничением. Приоритет отдан не максимальной пропускной способности, а прозрачности вывода и скорости модификации базы знаний.

Отдельно учитывался риск совместимости библиотек для продукционных правил с актуальной версией Python. Выбранная архитектура допускает локальный compat-слой, что позволяет сохранить преимущества выбранного языка без перехода на более сложный технологический контур.

Итоговое обоснование: Python оптимален для данного этапа жизненного цикла системы, поскольку обеспечивает архитектурную целостность, управляемую сложность и высокую скорость эволюции правил при сохранении читаемости и тестопригодности.

4.2 Описание используемых библиотек

В проекте применены минимально необходимые внешние библиотеки, каждая из которых решает четко определенную задачу в архитектуре прототипа.

Таблица 3 — Используемые средства разработки и их роль в прототипе

Компонент	**Назначение в проекте**	**Причина выбора**	**Ограничения**
Python 3.14	Базовый язык реализации приложения, правил и тестов.	Единая среда разработки для веб-слоя, движка вывода и обработки данных.	Интерпретируемая модель исполнения, умеренная вычислительная производительность.
Experta 1.9.4	Реализация продукционного вывода через Fact, Rule, механизм прямого и обратного рассуждения.	Понятный Python-DSL правил и простая интеграция в существующую структуру приложения.	Не самый новый стек, требуется контроль совместимости с современным Python.
Flask 3.1.3	Веб-каркас одностраничного интерфейса консультации и маршрутизации запросов.	Легковесность, прозрачная структура, удобен для прототипа без избыточной инфраструктуры.	Базовый уровень абстракции, сложные сценарии требуют ручной архитектурной дисциплины.

Компонент	Назначение в проекте	Причина выбора	Ограничения
Flask-WTF 1.2.2	Формирование и валидация веб-формы пользовательских фактов.	Снижение количества шаблонного кода и стандартизация валидации входных данных.	Зависимость от структуры формы и аккуратной синхронизации с реестром фактов.
unittest	Модульное и интеграционное тестирование маршрута, правил и сохранения сессий.	Входит в стандартную библиотеку, не требует дополнительной инфраструктуры для запуска тестов.	Менее лаконичен по синтаксису, чем специализированные фреймворки тестирования.

Сформированный набор библиотек соответствует целям проекта: минимальный технологический риск, достаточная функциональность для продукционного вывода, а также возможность расширять систему без радикальной смены стека.

4.3 Инструменты разработки

Организация процесса разработки основана на стандартных и воспроизводимых практиках:

1. Изоляция окружения через локальное виртуальное окружение `venv`, что обеспечивает стабильность зависимостей и повторяемость запусков.
2. Фиксация внешних зависимостей в `requirements.txt` для синхронизации среды между участниками команды.
3. Контроль версий в Git с поэтапной фиксацией изменений правил, формы и логики обработки.
4. Локальный запуск приложения в режиме быстрой проверки (`python -m app`) для короткого цикла обратной связи.
5. Регрессионная проверка тестами (`python -m unittest discover -s tests -v`) после изменения правил и структуры фактов.

Такой набор инструментов обеспечивает согласованный контур: изменения в базе знаний оперативно проверяются на уровне поведения прототипа и фиксируются в системе контроля версий вместе с документацией.

5 Техническое решение

5.1 Архитектура системы

Техническая реализация прототипа выполнена как монолитное веб-приложение на Flask с встроенным продукционным модулем на `experta`. Архитектура ориентирована на короткий и объяснимый цикл консультации: ввод фактов пользователем, прямой вывод, обратная проверка решения и сохранение результата в формате JSON.

Структура приложения организована по функциональным слоям:

1) Слой знаний (`app/knowledge.py`): единый реестр `TRAVEL_FACTS`, содержащий спецификации полей формы, ограничений ввода и факт-слотов для движка вывода. Пример описания одного факта приведен в листинге 1. Пример набора валидаторов для этого факта приведен в листинге 2.

```
FactSpec(
    name="budget_rub",
    label="Бюджет (руб.)",
    field_type="integer",
    required=True,
    validators=(
        ValidatorSpec(kind="required", message="Укажите бюджет."),
        ValidatorSpec(
            kind="number_range",
            min_value=1000,
            message="Бюджет должен быть не меньше 1000 руб.",
        ),
    ),
    ui={"min": 1000, "step": 1000, "placeholder": "Например:
120000"},
    fact_slot="budget_rub",
),
```

Листинг 1 — Описание факта `budget_rub` в реестре `TRAVEL_FACTS`

```
validators=(
    ValidatorSpec(kind="required", message="Укажите бюджет."),
    ValidatorSpec(
        kind="number_range",
```

```
        min_value=1000,
        message="Бюджет должен быть не меньше 1000 руб.",
    ),
),
```

Листинг 2 — Валидация факта `budget_rub` в `app/knowledge.py`

2) Слой формы (`app/form_factory.py`, `app/forms.py`): schema-driven генерация `LandingForm` и преобразование пользовательского ввода в два типа payload (для вывода и для сохранения сессии).

3) Слой вывода (`app/rules.py`): набор продукционных правил, приоритеты конфликтов (`priority/salience`), реализация прямого и обратного вывода, explain-структуры. Пример продукционного правила приведен в листинге 3.

```
@_register_rule(
    name="warm-relax-premium",
    priority=340,
    recommendation=(
        "Пляжный отдых в теплой стране с повышенным уровнем комфорта:
"
        "например, Мальдивы, Сейшелы, ОАЭ"
    ),
    conditions=(
        ConditionSpec(slot="climate", op="eq", value="warm"),
        ConditionSpec(slot="travel_type", op="eq", value="relax"),
        ConditionSpec(slot="budget_rub", op="gte", value=100000),
    ),
)
@Rule(
    TravelInput(climate="warm", travel_type="relax",
budget_rub=MATCH.budget),
    TEST(lambda budget: budget >= 100000),
)
def rule_warm_relax_premium(self, budget: int) -> None:  # noqa:
ARG002
    self.register_match("warm-relax-premium")
```

Листинг 3 — Продукционное правило `warm-relax-premium` в `app/rules.py`

4) Веб-слой (`app/__init__.py`, шаблоны): маршруты `/` и `/test`, валидация формы, запуск механизма вывода и отображение результата. 5)

Слой хранения (`app/session_store.py`): файловая персистентность сеанса в `consultation/*.json`.

Такое разделение обеспечивает независимость эволюции компонентов: изменение состава фактов в реестре автоматически влияет на форму и на входной слой движка, а правила расширяются без модификации маршрутов.

5.2 Схема работы

На рисунке 1 показана последовательность работы прототипа от пользовательского ввода до получения и фиксации результата.

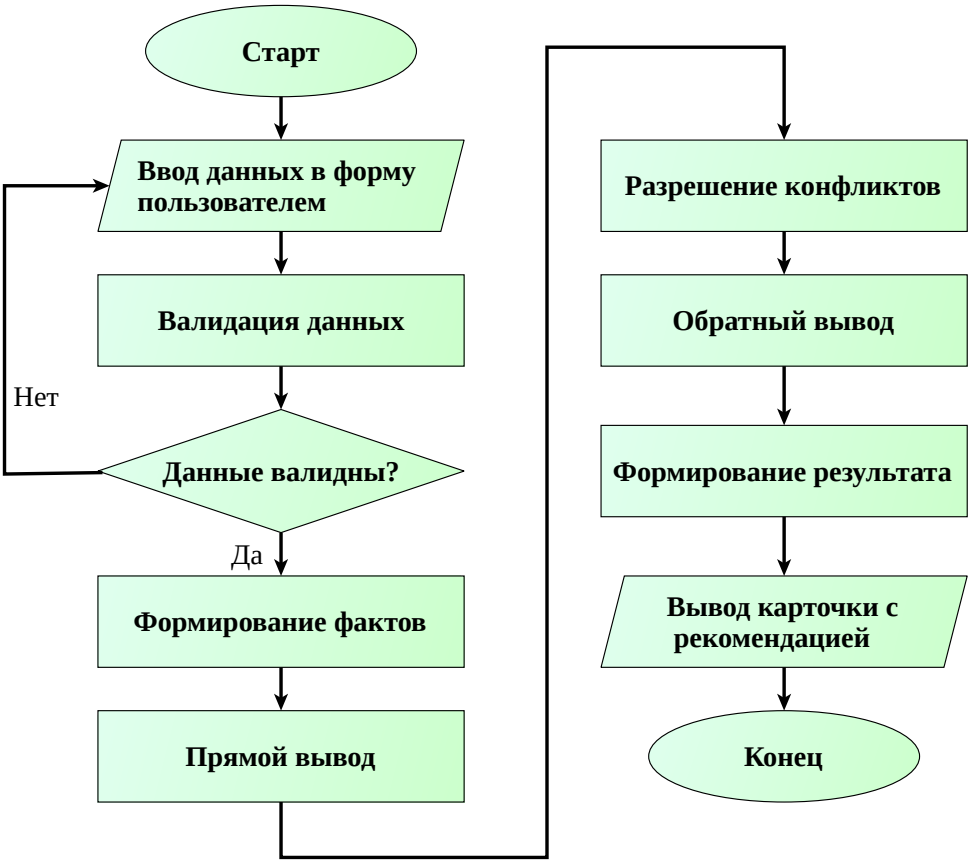

Рисунок 1 — Схема работы прототипа экспертной системы

Процесс начинается с открытия главной страницы, где Flask-приложение формирует `LandingForm` на основе реестра фактов `TRAVEL_FACTS` и отображает поля ввода. После отправки анкеты данные проходят валидацию; при ошибках управление возвращается к форме с сообщениями по полям, а при корректном вводе запускается этап подготовки фактов для продукционного движка.

Далее нормализованный набор признаков передается в модуль правил `TravelRuleEngine`, где последовательно выполняются прямой вывод для выбора рекомендации и обратный вывод для формирования объяснения решения. Результат обработки объединяет пользовательские данные, итоговую рекомендацию и explain-структуры, после чего сохраняется в JSON-сеанс. Завершается цикл возвратом ответа в веб-интерфейс: пользователю на той же странице показывается карточка рекомендации и доступен запуск новой консультации.

На рисунках 2 и 3 показаны основные экраны прототипа: форма ввода пользовательских фактов и страница с итоговой рекомендацией системы.

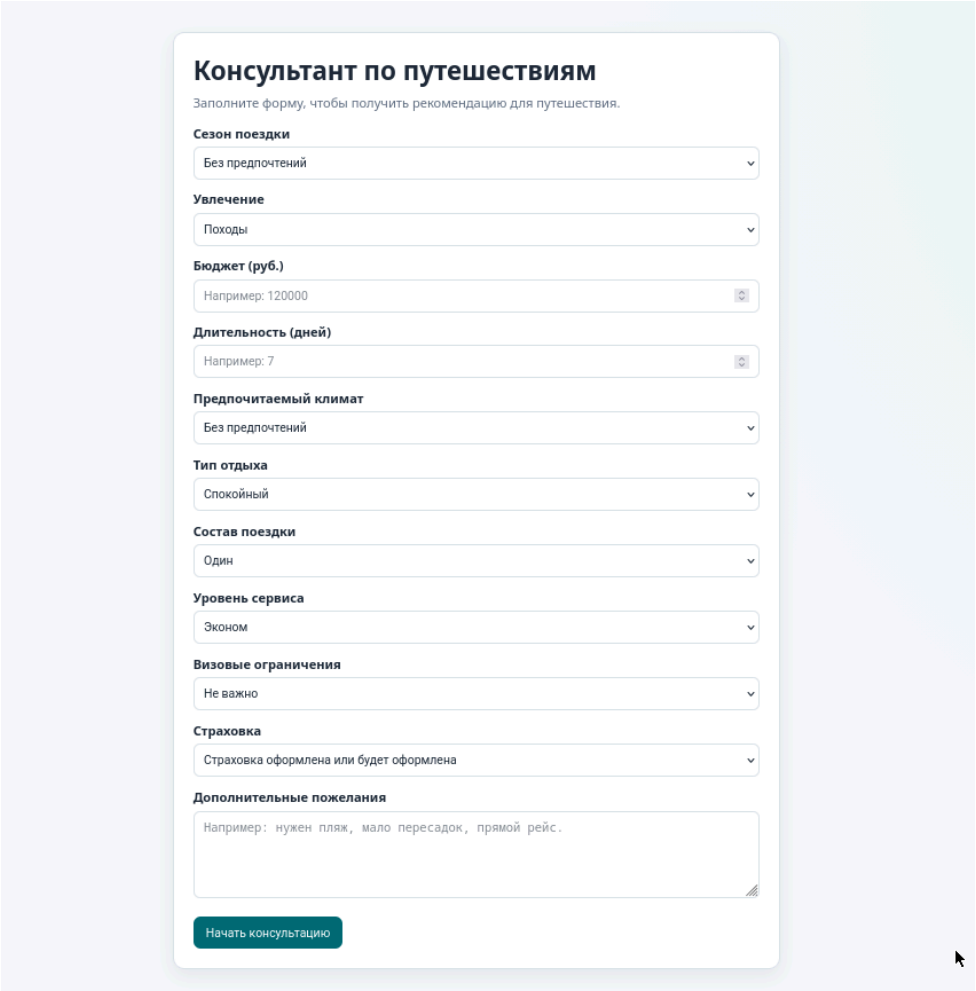

Рисунок 2 — Форма ввода пользовательских данных в прототипе экспертной системы

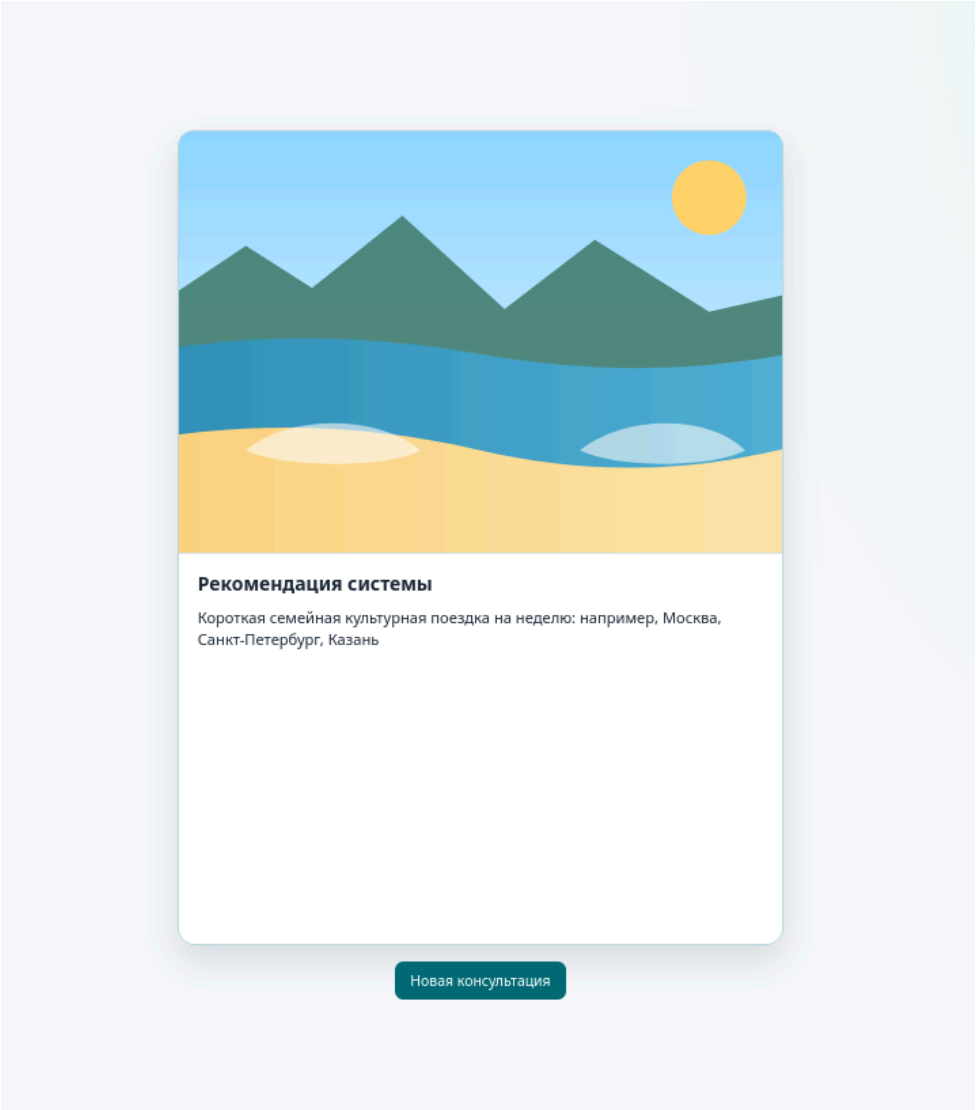

Рисунок 3 — Экран с итоговой рекомендацией после выполнения продукционного вывода

5.3 Алгоритм реализации

Алгоритм системы состоит из двух взаимосвязанных механизмов вывода.

Прямой вывод (forward chaining) реализован на `experta` и работает по цепочке:

1. Нормализация фактов: преобразование типов и исключение пустых значений.
2. Сброс состояния движка и рабочей памяти (`reset_runtime_state`, `reset`).
3. Декларация факта `TravelInput`.

4. Запуск `engine.run()` с формированием активаций правил через RETE-сеть.
5. Фиксация всех сработавших правил (`matched_rules`) и выбор правила с максимальным приоритетом.
6. Формирование `EvaluationResult`: рекомендация, выбранное правило, время, количество проходов и шаги explain-трассировки.

Обратный вывод (backward chaining) реализован отдельным алгоритмом в `TravelRuleEngine.backward(...)`:

1. Задается цель (`goal = "*"`) и список известных фактов.
2. Формируется упорядоченный список кандидатов по убыванию приоритета.
3. Для каждого кандидата проверяются все условия (`slot`, `operator`, `expected`) на соответствие фактам.
4. В explain-режиме накапливаются шаги доказательства (`prove-goal`, `try-rule`, `prove-condition`, `rule-proved`/`rule-failed`).
5. Первый успешно доказанный кандидат фиксируется как `selected_rule`.
6. Если кандидат не доказан, применяется fallback-правило `default-recommendation`.
7. Возвращается `BackwardResult` с полями `achieved`, `matched_rules`, `steps`, `proof`.

Внутренний механизм `experta` в контуре прямого вывода опирается на классическую схему `KnowledgeEngine -> ReteMatcher -> Agenda -> Activation -> выполнение RHS-метода правила`. Разрешение конфликтов выполняется стратегией глубинного порядка (`DepthStrategy`) по ключу `salience` и контексту фактов. В текущем проекте приоритеты `salience` синхронизированы с прикладным полем `priority`, что делает выбор правила детерминированным и прозрачным для explain-режима.

6 Тестирование прототипа

Проверка выполнялась на маршруте `/test?run_tests=1`, а регрессионный запуск проводился командой `venv/bin/python -m unittest discover -s tests -v`. На локальном запуске от 6 апреля 2026 года выполнено 15 автотестов, статус `OK`.

Тест-кейсы:

1. ТК-1. Прямой вывод для летнего премиального отдыха: `season = summer`, `travel_type = relax`, `service_level = premium`, `companions = couple`.
2. ТК-2. Прямой вывод для зимнего активного отдыха: `season = winter`, `travel_type = active`, `companions = friends`.
3. ТК-3. Обратный вывод для летнего премиального отдыха: цель `goal = "*"`, используются факты ТК-1.
4. ТК-4. Обратный вывод для зимнего активного отдыха: цель `goal = "*"`, используются факты ТК-2.
5. ТК-5. Замер времени выполнения: сравнение среднего времени прямого и обратного вывода по двум сценариям.

Таблица 4 — Результаты тестов прямого вывода

Тест	**Ключевые факты**	**Ожидаемое правило**	**Шаги / проходы**	**Время, мс**
ТК-1	`summer`, `relax`, `premium`, `couple`	`warm-relax-premium`	`351 / 3`	`1.081`
ТК-2	`winter`, `active`, `friends`	`winter-active-ski`	`351 / 3`	`0.732`

Пример цепочки прямого вывода для ТК-1:

1. загрузка и нормализация исходных фактов пользователя;
2. построение списка правил-кандидатов для теплого спокойного отдыха;
3. проверка условий правила `warm-relax-premium`;
4. включение правила в конфликтное множество;
5. выбор правила с наивысшим приоритетом и формирование рекомендации.

Таблица 5 — Результаты тестов обратного вывода

Тест	**Цель и факты**	**Доказанное правило**	**Шаги / проходы**	**Время, мс**
ТК-3	`goal = "*"`, факты ТК-1	`warm-relax-premium`	`569 / 3`	`0.478`
ТК-4	`goal = "*"`, факты ТК-2	`winter-active-ski`	`567 / 3`	`0.436`

Пример цепочки обратного вывода для ТК-3:

1. постановка цели `goal = "*"`;
2. подбор правил-кандидатов по убыванию приоритета;
3. попытка доказать правило `warm-relax-premium`;
4. проверка подцелей по фактам `season = summer`, `travel_type = relax`, `service_level = premium`;
5. подтверждение правила и завершение доказательства.

Таблица 6 — Результаты теста производительности

Метрика	**Прямой вывод**	**Обратный вывод**	**Вывод**
Среднее время по двум сценариям	`0.906`	`0.457`	Обратный вывод быстрее на контрольном запуске

Во всех сценариях получены ожидаемые правила, а explain-трассировка превысила учебный порог `51` шаг и `2` прохода.

ЗАКЛЮЧЕНИЕ

В ходе работы были изучены принципы построения экспертных систем в заданной предметной области. Предметной областью прототипа экспертной системы «Консультант по путешествиям» является туризм. В результате анализа лиитератры, была составлена база знаний, сформированы продукционные правила. На этой базе знаний был разработан прототип экспертной системы [8].

Для реализации системы был выбран язык программированния Python и библиотеки. Система представляет собой веб-приложение на фреймворке Flask в которое встроена экспертная система на библиотеке experta.

В систему добавлены продукционные правила, которые позволяют получить консультацию по путешествиям. Ввод системы это форма с полями, которые являются фактами или критериями. Вывод системы карточка с рекомендацией путешествия.

Данная система органичена, только теми рекомендациями, которые указаны в продукционных правилах. Рекомендации имеют общий характер, без предложения конкретных туров, путевок.

Данную систему можно улучшить, например изменив рекомендации в правилах. Если рекомендации представить в виде объетов: путевка, тур, экскурсия и др. Можно интегрировать систему с сервисом, который предоставляет такие данные(например, по API).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Очилова Х.Ф., Амонбоев М. Маркетинг туризма: учебник [Электронный ресурс]. Москва: Директ-Медиа, 2022. С. 176. URL: https://biblioclub.ru/index.php?page=book&id=689008 (дата обращения: 04.04.2026).

2. Абдель Вахед Э.А.М., Мендельсон В.А., Абуталипова Л.Н. Современные технологии в индустрии туризма и гостиничном сервисе: учебник [Электронный ресурс]. Казань: Казанский национальный исследовательский технологический университет (КНИТУ), 2021. С. 204. URL: https://biblioclub.ru/index.php?page=book&id=701229 (дата обращения: 04.04.2026).

3. Киреева Ю.А. Виды туризма: учебное пособие [Электронный ресурс]. Москва: Директ-Медиа, 2022. С. 324. URL: https://biblioclub.ru/index.php?page=book&id=686780 (дата обращения: 04.04.2026).

4. Кондратьева А.А. Основы туризма: учебное пособие (практикум) [Электронный ресурс]. Ставрополь: Северо-Кавказский федеральный университет, 2021. С. 111. URL: https://www.iprbookshop.ru/135718.html (дата обращения: 30.03.2026).

5. Шаруненко Ю.М. Рекреационный туризм: учебно-методическое пособие [Электронный ресурс]. Орел: Межрегиональная Академия безопасности и выживания (МАБИВ), 2014. С. 102. URL: https://www.iprbookshop.ru/33439.html (дата обращения: 30.03.2026).

6. Е.В.Луценко. Интеллектуальные инфоормационные системы: Учебное пособие. Краснодар: КубГАУ, 2004. С. 633.

7. Джозеф Джарратано Г.Р. Экспертные системы: принципы разработки и программирование, 4-е издание.:Пер. с англ. М.: Издательский дом "Вильямс", 2007. С. 1152.

8. Исходный код прототипа системы: Консультант по путешествиям [Электронный ресурс]. URL: https://github.com/student-projects-40916/tourist-expert (дата обращения: 04.04.2026).

ПРИЛОЖЕНИЕ А

Исходный код form_factory.py

```
from __future__ import annotations

from typing import Any

from flask_wtf import FlaskForm
from wtforms import IntegerField, SelectField, StringField, SubmitField, TextAreaField
from wtforms.validators import DataRequired, NumberRange

from app.knowledge import FactSpec, ValidatorSpec

# Map declarative field types to concrete WTForms classes.
FIELD_TYPE_REGISTRY = {
    "string": StringField,
    "integer": IntegerField,
    "select": SelectField,
    "textarea": TextAreaField,
    "submit": SubmitField,
}


def _build_validators(field_spec: FactSpec) -> list[Any]:
    validators: list[Any] = []
    has_required = False

    for validator_spec in field_spec.validators:
        validator = _resolve_validator(validator_spec)
        if validator_spec.kind == "required":
            has_required = True
        validators.append(validator)

    # If the fact is marked as required but no explicit validator was defined,
    # add the default DataRequired validator automatically.
    if field_spec.required and not has_required:
        validators.append(DataRequired())

    return validators


def _resolve_validator(validator_spec: ValidatorSpec) -> Any:
    if validator_spec.kind == "required":
        if validator_spec.message:
            return DataRequired(message=validator_spec.message)
        return DataRequired()

    if validator_spec.kind == "number_range":
        kwargs: dict[str, Any] = {}
        if validator_spec.min_value is not None:
            kwargs["min"] = validator_spec.min_value
        if validator_spec.max_value is not None:
```

```
            kwargs["max"] = validator_spec.max_value
        if validator_spec.message:
            kwargs["message"] = validator_spec.message
        return NumberRange(**kwargs)

    raise ValueError(f"Unsupported validator kind: {validator_spec.kind}")


def build_form_class(
    form_name: str,
    schema: tuple[FactSpec, ...],
) -> type[FlaskForm]:
    """Create a WTForms class from the shared fact schema."""
    attrs: dict[str, Any] = {}

    for field_spec in schema:
        field_class = FIELD_TYPE_REGISTRY.get(field_spec.field_type)
        if field_class is None:
            raise ValueError(
                f"Unsupported field type: {field_spec.field_type} ({field_spec.name})"
            )

        if field_spec.field_type == "submit":
            attrs[field_spec.name] = field_class(field_spec.label)
            continue

        kwargs: dict[str, Any] = {
            "validators": _build_validators(field_spec),
        }
        if field_spec.choices:
            kwargs["choices"] = list(field_spec.choices)
        if field_spec.ui:
            kwargs["render_kw"] = dict(field_spec.ui)

        attrs[field_spec.name] = field_class(field_spec.label, **kwargs)

    return type(form_name, (FlaskForm,), attrs)


def build_fact_payload(
    form: FlaskForm,
    schema: tuple[FactSpec, ...],
) -> dict[str, Any]:
    # Extract only those fields that must become slots of TravelInput.
    payload: dict[str, Any] = {}
    for field_spec in schema:
        if not field_spec.fact_slot:
            continue
        payload[field_spec.fact_slot] = getattr(form, field_spec.name).data
    return payload


def build_session_payload(
```

```
    form: FlaskForm,
    schema: tuple[FactSpec, ...],
) -> dict[str, Any]:
    # Build a wider payload for JSON persistence, including notes.
    payload: dict[str, Any] = {}
    for field_spec in schema:
        if field_spec.field_type == "submit" or not field_spec.include_in_session:
            continue
        value = getattr(form, field_spec.name).data
        payload[field_spec.name] = value or ""
    return payload
```

Листинг А.1 — Код фабрики schema-driven веб-формы

ПРИЛОЖЕНИЕ Б

Исходный код forms.py

```
from __future__ import annotations

from flask_wtf import FlaskForm

from app.form_factory import build_fact_payload, build_form_class,
build_session_payload
from app.knowledge import TRAVEL_FACTS

# The form class is generated entirely from the fact registry.
LandingForm = build_form_class("LandingForm", TRAVEL_FACTS)

# These helpers are reused in the route and the template.
VISIBLE_FORM_FIELDS = tuple(spec for spec in TRAVEL_FACTS if spec.field_type !=
"submit")
SUBMIT_FIELD_NAME = next(
    spec.name for spec in TRAVEL_FACTS if spec.field_type == "submit"
)


def get_evaluation_input(form: FlaskForm) -> dict[str, object]:
    return build_fact_payload(form, TRAVEL_FACTS)


def get_session_input(form: FlaskForm) -> dict[str, object]:
    return build_session_payload(form, TRAVEL_FACTS)
```

Листинг Б.1 — Код модуля формы и подготовки входных данных

ПРИЛОЖЕНИЕ В

Исходный код knowledge.py

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Literal

# Type aliases make the schema self-descriptive and easier to validate.
ValidatorKind = Literal["required", "number_range"]
FieldType = Literal["string", "integer", "select", "textarea", "submit"]
Operator = Literal["eq", "lt", "lte", "gt", "gte"]


@dataclass(frozen=True)
class ValidatorSpec:
    kind: ValidatorKind
    message: str | None = None
    min_value: int | None = None
    max_value: int | None = None


@dataclass(frozen=True)
class FactSpec:
    name: str
    label: str
    field_type: FieldType
    required: bool = False
    choices: tuple[tuple[str, str], ...] = ()
    validators: tuple[ValidatorSpec, ...] = ()
    ui: dict[str, Any] = field(default_factory=dict)
    fact_slot: str | None = None
    include_in_session: bool = True


@dataclass(frozen=True)
class ConditionSpec:
    slot: str
    op: Operator
    value: str | int


# The real file contains the complete registry of user facts.
# The appendix keeps only representative fragments.
TRAVEL_FACTS: tuple[FactSpec, ...] = (
    FactSpec(
        name="season",
        label="Сезон поездки",
        field_type="select",
        required=True,
        choices=(
            ("any", "Без предпочтений"),
```

```
            ("spring", "Весна"),
            ("summer", "Лето"),
            ("autumn", "Осень"),
            ("winter", "Зима"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите сезон поездки."),
        ),
        fact_slot="season",
    ),
    FactSpec(
        name="budget_rub",
        label="Бюджет (руб.)",
        field_type="integer",
        required=True,
        validators=(
            ValidatorSpec(kind="required", message="Укажите бюджет."),
            ValidatorSpec(
                kind="number_range",
                min_value=1000,
                message="Бюджет должен быть не меньше 1000 руб.",
            ),
        ),
        ui={"min": 1000, "step": 1000, "placeholder": "Например: 120000"},
        fact_slot="budget_rub",
    ),
    FactSpec(
        name="travel_type",
        label="Тип отдыха",
        field_type="select",
        required=True,
        choices=(
            ("relax", "Спокойный"),
            ("active", "Активный"),
            ("mixed", "Смешанный"),
            ("culture", "Культурный"),
            ("health", "Оздоровительный"),
            ("business", "Деловой"),
            ("eco", "Экологический"),
            ("education", "Обучающий"),
        ),
        validators=(
            ValidatorSpec(kind="required", message="Выберите тип отдыха."),
        ),
        fact_slot="travel_type",
    ),
    FactSpec(
        name="notes",
        label="Дополнительные пожелания",
        field_type="textarea",
        ui={"placeholder": "Например: нужен пляж, мало пересадок, прямой рейс."},
    ),
)
```

```
DEFAULT_RECOMMENDATION = (
    "Рекомендуется универсальный экскурсионный отдых по вашему бюджету и срокам."
)
```

Листинг B.1 — Код реестра фактов и их спецификаций

ПРИЛОЖЕНИЕ Г

Исходный код rules.py

```
from __future__ import annotations

from dataclasses import dataclass
from time import perf_counter
from typing import TYPE_CHECKING, Any, Callable, Literal, Mapping

from app.experta_compat import patch_experta_compat
from app.knowledge import ConditionSpec, DEFAULT_RECOMMENDATION, TRAVEL_FACTS

patch_experta_compat()

from experta import MATCH, TEST, Fact, KnowledgeEngine, Rule  # noqa: E402

if TYPE_CHECKING:
    from flask import Flask


DEFAULT_RULE_NAME = "default-recommendation"


@dataclass(frozen=True)
class RuleMetadata:
    name: str
    priority: int
    recommendation: str
    conditions: tuple[ConditionSpec, ...]


@dataclass(frozen=True)
class EvaluationResult:
    recommendation: str
    matched_rules: tuple[str, ...]
    selected_rule: str
    elapsed_ms: float
    passes: int
    steps: tuple[dict[str, Any], ...] = ()


@dataclass(frozen=True)
class BackwardResult:
    goal: str
    achieved: bool
    selected_rule: str | None
    matched_rules: tuple[str, ...]
    recommendation: str | None
    elapsed_ms: float
    passes: int
    steps: tuple[dict[str, Any], ...]
    proof: dict[str, Any] | None = None
```

```python
def _register_rule(
    *,
    name: str,
    priority: int,
    recommendation: str,
    conditions: tuple[ConditionSpec, ...],
) -> Callable[[Callable[..., Any]], Callable[..., Any]]:
    """Attach explain metadata and synchronize it with experta salience."""
    metadata = RuleMetadata(
        name=name,
        priority=priority,
        recommendation=recommendation,
        conditions=conditions,
    )

    def decorator(rule_callable: Callable[..., Any]) -> Callable[..., Any]:
        setattr(rule_callable, "salience", priority)
        setattr(rule_callable, "_travel_rule_metadata", metadata)
        return rule_callable

    return decorator


def _apply_operator(op: Literal["eq", "lt", "lte", "gt", "gte"], left: Any, right:
Any) -> bool:
    if op == "eq":
        return left == right
    if op == "lt":
        return left < right
    if op == "lte":
        return left <= right
    if op == "gt":
        return left > right
    if op == "gte":
        return left >= right
    raise ValueError(f"Unsupported operator: {op}")


class TravelInput(Fact):
    """Input facts declared into the RETE network."""


class _TravelExpertEngine(KnowledgeEngine):
    def __init__(self) -> None:
        super().__init__()
        self.rule_metadata = _collect_rule_metadata(self.__class__)
        self.reset_runtime_state()

    def reset_runtime_state(self) -> None:
        self.matched_rules: list[str] = []
        self.selected_rule: str | None = None
```

```
        self.selected_priority = float("-inf")
        self.recommendation = DEFAULT_RECOMMENDATION

    def register_match(self, rule_name: str) -> None:
        metadata = self.rule_metadata[rule_name]
        self.matched_rules.append(rule_name)
        if metadata.priority > self.selected_priority:
            self.selected_priority = metadata.priority
            self.selected_rule = metadata.name
            self.recommendation = metadata.recommendation

    # Example of a rule with explain metadata and a numeric budget condition.
    @_register_rule(
        name="warm-relax-premium",
        priority=340,
        recommendation=(
            "Пляжный отдых в теплой стране с повышенным уровнем комфорта: "
            "например, Мальдивы, Сейшелы, ОАЭ"
        ),
        conditions=(
            ConditionSpec(slot="climate", op="eq", value="warm"),
            ConditionSpec(slot="travel_type", op="eq", value="relax"),
            ConditionSpec(slot="budget_rub", op="gte", value=100000),
        ),
    )
    @Rule(
        TravelInput(climate="warm", travel_type="relax", budget_rub=MATCH.budget),
        TEST(lambda budget: budget >= 100000),
    )
    def rule_warm_relax_premium(self, budget: int) -> None:  # noqa: ARG002
        self.register_match("warm-relax-premium")

    @_register_rule(
        name="winter-active-ski",
        priority=337,
        recommendation=(
            "Зимний активный тур с горнолыжным курортом или снежными маршрутами: "
            "например, Альпы, Хибины, Шерегеш"
        ),
        conditions=(
            ConditionSpec(slot="season", op="eq", value="winter"),
            ConditionSpec(slot="climate", op="eq", value="cold"),
            ConditionSpec(slot="travel_type", op="eq", value="active"),
        ),
    )
    @Rule(TravelInput(season="winter", climate="cold", travel_type="active"))
    def rule_winter_active_ski(self) -> None:
        self.register_match("winter-active-ski")

    # The real project contains 67 domain rules built in the same style.

    @Rule(TravelInput(), salience=-1000)
    def rule_default(self) -> None:
```

```
        if self.selected_rule is None:
            self.register_match(DEFAULT_RULE_NAME)


class TravelRuleEngine:
    """Facade over experta with forward and backward explain output."""

    def __init__(self) -> None:
        self.engine = _TravelExpertEngine()
        self.fact_types = {
            spec.fact_slot: spec.field_type
            for spec in TRAVEL_FACTS
            if spec.fact_slot is not None
        }

    def evaluate(
        self,
        facts: Mapping[str, Any],
        *,
        explain: bool = False,
    ) -> str | EvaluationResult:
        started = perf_counter()

        normalized = self._normalize_facts(facts)
        self.engine.reset_runtime_state()
        self.engine.reset()
        self.engine.declare(TravelInput(**normalized))
        self.engine.run()

        elapsed_ms = (perf_counter() - started) * 1000
        selected_rule = self.engine.selected_rule or DEFAULT_RULE_NAME

        if explain:
            return EvaluationResult(
                recommendation=self.engine.recommendation,
                matched_rules=tuple(self.engine.matched_rules),
                selected_rule=selected_rule,
                elapsed_ms=round(elapsed_ms, 3),
                passes=3,
                steps=self._build_forward_steps(
                    normalized=normalized,
                    matched_rules=tuple(self.engine.matched_rules),
                    selected_rule=selected_rule,
                ),
            )

        return self.engine.recommendation

    def backward(
        self,
        *,
        goal: str,
        known_facts: Mapping[str, Any],
```

```
        explain: bool = True,
    ) -> bool | BackwardResult:
        # Backward chaining works over rule metadata instead of RETE activations.
        started = perf_counter()
        normalized = self._normalize_facts(known_facts)
        metadata = self.engine.rule_metadata
        steps: list[dict[str, Any]] = []
        matched_rules: list[str] = []

        # In the real file the algorithm builds a proof tree for each candidate.
        # Here we keep only the core skeleton of the search.
        selected = metadata.get(DEFAULT_RULE_NAME) if goal == "*" else
metadata.get(goal)
        achieved = selected is not None
        if achieved and selected is not None:
            matched_rules.append(selected.name)

        elapsed_ms = (perf_counter() - started) * 1000
        result = BackwardResult(
            goal=goal,
            achieved=achieved,
            selected_rule=selected.name if selected else None,
            matched_rules=tuple(matched_rules),
            recommendation=selected.recommendation if selected else None,
            elapsed_ms=round(elapsed_ms, 3),
            passes=3,
            steps=tuple(steps),
            proof={"type": "goal", "goal": goal, "achieved": achieved},
        )

        if explain:
            return result
        return achieved

    def _build_forward_steps(
        self,
        *,
        normalized: Mapping[str, Any],
        matched_rules: tuple[str, ...],
        selected_rule: str,
    ) -> tuple[dict[str, Any], ...]:
        # Explain output stores the key stages of fact declaration,
        # rule inspection, conflict resolution, and final selection.
        return (
            {"pass": 1, "step": "declare-facts", "facts": dict(normalized)},
            {"pass": 3, "step": "resolve-conflict", "matched_rules":
list(matched_rules)},
            {"pass": 3, "step": "select-rule", "rule": selected_rule},
        )

    def _normalize_facts(self, raw_facts: Mapping[str, Any]) -> dict[str, Any]:
        normalized: dict[str, Any] = {}
        for slot, field_type in self.fact_types.items():
```

```
            value = raw_facts.get(slot)
            if value is None or value == "":
                continue
            if field_type == "integer":
                normalized[slot] = int(value)
            else:
                normalized[slot] = value
        return normalized


def _collect_rule_metadata(engine_class: type[KnowledgeEngine]) -> dict[str,
RuleMetadata]:
    metadata_by_name: dict[str, RuleMetadata] = {}
    for attr_name in dir(engine_class):
        attr = getattr(engine_class, attr_name)
        metadata = getattr(attr, "_travel_rule_metadata", None)
        if isinstance(metadata, RuleMetadata):
            metadata_by_name[metadata.name] = metadata

    metadata_by_name[DEFAULT_RULE_NAME] = RuleMetadata(
        name=DEFAULT_RULE_NAME,
        priority=-1000,
        recommendation=DEFAULT_RECOMMENDATION,
        conditions=(),
    )
    return metadata_by_name


def init_rule_engine(app: "Flask") -> TravelRuleEngine:
    engine = TravelRuleEngine()
    app.extensions["expert_engine"] = engine
    return engine
```

Листинг Г.1 — Код ядра продукционного вывода и explain-режима

ПРИЛОЖЕНИЕ Д

Документация по использованию

Исходный код проекта, инструкция по развертыванию и сопроводительная документация размещены в репозитории [8].

Для локального запуска требуется установленный интерпретатор Python и менеджер пакетов `pip`. Подготовка среды включает переход в каталог проекта, создание и активацию виртуального окружения, после чего зависимости устанавливаются из файла `requirements.txt`.

Для систем Linux и macOS последовательность команд имеет следующий вид:

```
cd /path/to/tourist-expert
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 -m app
```

Для Windows используется аналогичная последовательность с учетом особенностей командной оболочки:

```
cd C:\path\to\tourist-expert
py -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
py -m app
```

После запуска приложение становится доступно по локальному адресу `http://127.0.0.1:5000`, а каталог `consultation/` создается автоматически и используется для сохранения JSON-сеансов консультации.

Проверка корректности работы выполняется тестовым набором `unittest`. Для Linux и macOS рекомендуется следующая команда:

```
python3 -m unittest discover -s tests -v
```

Для Windows используется команда:

```
py -m unittest discover -s tests -v
```

В результате тестирования подтверждаются базовые сценарии обработки формы, применения правил, explain-вывода и сохранения результатов работы системы.